

## **Лабораторная работа №25**

### **Изучение процесса отрисовки графики**

#### **1 Цель работы**

1.1 Научиться разрабатывать пользовательские графические элементы в оконных приложениях.

#### **2 Литература**

2.1 Фленов, М. Е. Библия C#. 4 изд / М. Е. Фленов. – Санкт-Петербург: БХВПетербург, 2019. – 512 с. – URL: <https://ibooks.ru/bookshelf/366634/reading>. – Режим доступа: только для зарегистрированных пользователей. – Текст : электронный. – гл.13.

#### **3 Подготовка к работе**

3.1 Повторить теоретический материал (см. п.2).

3.2 Изучить описание лабораторной работы.

#### **4 Основное оборудование**

4.1 Персональный компьютер.

#### **5 Задание**

5.1 Создание нового проекта

5.1.1 Создать новый проект WPF.

5.1.2 Добавить в проект новый класс CustomButton, который необходимо сделать наследником FrameworkElement

5.1.3 Добавить в класс поля и свойства необходимые для отрисовки:

```
private VisualCollection _children;
private DrawingVisual _visual;
protected override int VisualChildrenCount => _children.Count;
protected override Visual GetVisualChild(int index) => _children[index];
```

5.1.4 Инициализировать свойства в конструкторе класса:

```
_children = new VisualCollection(this);
_visual = new DrawingVisual();
_children.Add(_visual);
```

5.1.5 В конструкторе добавить подписку на событие и создайте новый пустой обработчик OnRenderFrame:

```
CompositionTarget.Rendering += OnRenderFrame;
```

5.2 Отрисовка основы кнопки

5.2.1 Добавить в класс новый приватный метод Draw, добавить вызов метода Draw в обработчик OnRenderFrame.

5.2.2 В методе Draw необходимо получить объект DrawingContext, необходимый для отрисовки графики.

```
private void Draw()
{
    using (var dc = _visual.RenderOpen())
    {
        // отрисовка
    }
}
```

5.2.3 Для отрисовки базы кнопки используем метод `dc.DrawRoundedRectangle`: первым параметром в метод передается объект `SolidColorBrush` для цвета фона, вторым параметром передается объект `Pen` для цвета и толщины обводки, третьим – объект `Rect`, определяющий размеры и позицию отрисовываемого элемента, четвертый и пятый – радиусы скругления прямоугольника по горизонтали и вертикали, например:

```
dc.DrawRoundedRectangle(
    new SolidColorBrush(Colors.DodgerBlue),
    null,
    new Rect(0, 0, Width, Height),
    10, 10);
```

5.2.4 Добавить отрисовку тени для элемента аналогичным методом, указав полупрозрачный цвет для фона и некоторое небольшое смещение для координат `Rect`. (отрисовка тени должна выполняться до отрисовки самого объекта)

### 5.3 Отрисовка текста на кнопке

5.3.1 Для подготовки текста перед отрисовкой необходимо обработать его через `FormattedText`, например

```
var text = new FormattedText(
    "Button", // текст
    System.Globalization.CultureInfo.CurrentCulture, // информация о культуре
    FlowDirection.LeftToRight, // направление текста
    new Typeface("Segoe UI"), // шрифт
    20, // размер шрифта
    Brushes.White, // цвет текста
    1.25); // значение, зависящее от DPI
```

5.3.2 Далее необходимо отрисовать текст при помощи `DrawText`, позицию текста вычислять относительно ширины и высоты кнопки и форматированного текста

```
dc.DrawText(text, // FormattedText
    textPos); // позиция текста, объект Point
```

### 5.4 Анимация при нажатии и наведении

5.4.1 Добавить в класс поля для хранения состояний наведения и нажатия на кнопку, а также для скейлинга кнопки:

```
private bool _isHovered;
private bool _isPressed;

private double _scale = 1.0;
private double _targetScale = 1.0;
```

5.4.2 В конструкторе класса добавить обработчики для событий `MouseEnter`, `MouseLeave`, `MouseDown`, `MouseUp`:

При наведении на кнопку `targetScale` устанавливать в 110%, когда кнопка нажата – в 95%, так же при срабатывании данных событий необходимо менять флаги `isHovered` и `isPressed`

5.4.3 При отрисовке кнопки менять цвет фона кнопки в зависимости от установленных флагов

5.4.4 Для изменения размера кнопки при нажатии добавить в метод `Draw` код, добавляющий трансформацию графики:

```
dc.PushTransform(new ScaleTransform(_scale, _scale, center.X, center.Y));

// вся предыдущая логика отрисовки графики
```

```
dc.Pop();
```

center – объект Point, который необходимо вычислять относительно ширины и высоты кнопки.

5.4.5 В обработчик OnRenderFrame добавить код для плавной смены размера перед вызовом метода Draw:

```
_scale += (_targetScale - _scale) * 0.15; // плавная интерполяция
```

## 5.5 Обработка нажатий

5.5.1 Добавить в класс кнопки RoutedEvent для обработки нажатия на кнопку:

```
public static readonly RoutedEvent ClickEvent =
   EventManager.RegisterRoutedEvent(
        "Click",
        RoutingStrategy.Bubble,
        typeof(RoutedEventHandler),
        typeof(AnimatedButton));

public event RoutedEventHandler Click
{
    add { AddHandler(ClickEvent, value); }
    remove { RemoveHandler(ClickEvent, value); }
}
```

5.5.2 В обработчике MouseUp вызывать событие Click, если кнопка была нажата и курсор находится над кнопкой

```
RaiseEvent(new RoutedEventArgs(ClickEvent, this));
```

## 5.6 Доработка кнопки.

5.6.1 Вынести основные данные, заданные в коде и используемые при отрисовке, например текст, цвета и т. д. в публичные свойства, что бы пользователь мог их настроить в разметке.

5.6.2 Протестировать кнопку с разными параметрами в приложении.

## 6 Порядок выполнения работы

6.1 Запустить MS Visual Studio и создать оконное приложение WPF C#.

6.2 Выполнить все задания из п.5 в одном решении.

6.3 Ответить на контрольные вопросы.

## 7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

## 8 Контрольные вопросы

8.1 Для чего используется класс SolidColorBrush?

8.2 Для чего используется класс Pen?

8.3 Какие основные возможности предоставляет класс DrawingContext?